CSE233 Week 8: Arrays

This week, we will discuss arrays. Arrays are a fundamental data structure in computer science and programming, and are implemented in nearly all programming languages. Arrays are fixed-size containers that store elements of the same data types. They allow a single name to be used to access multiple data values. Individual values are accessed via a subscript (also called index).

Declaring and Using Arrays

Below is the declaration of an array of 10 integers in C++:

```
int array[10];
```

First, the data type is specified. Then, the name of the array. After the name is a set of square brackets with the number of elements in the array. An initializer list can be used to provide initial values to the array. An initializer list is a set of curly-braces with a comma-separated list of values inside. When using an initializer list, the size of the array can be deduced by the compiler:

```
// still has 10 elements
int array[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

The individual elements of the array are accessed via the subscript operator. The subscript operator is a set of square brackets placed after the array name. The square brackets contain an index, which is an integer which specifies which array element is being accessed. Arrays in C++ are 0-indexed, meaning the first element has index 0, the second element has index 1, etc. The subscript operator can be used to both read and write values.

```
int array[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

// array[0] is the first element
```

```
assert(array[0] == 1);
// array[9] is the last element
assert(array[9] == 10);

// we can use a loop to print every element in the array
for (int i = 0; i < 10; ++i) {
    std::cout << "array[" << i << "] = " << array[i] << "\n";
}

// we can also use the index to change the values of
// the array's elements
array[0] = 10;
array[9] = 1;

// change them back
array[0] = 1;
array[9] = 10;

// we can also do so in a loop
for (int i = 0; i < 10; ++i) {
    array[i] = array[i] * 2;
}
// array is now:
// { 2, 4, 6, 8, 10, 12, 14, 16, 18, 20 }
```

As noted in the example, the last element of an array with 10 elements is indexed with 9. This is due to the array being 0-indexed; the last element is always one less than the number of elements it contains.

To understand why arrays are indexed the way they are, let's look at how arrays are stored in memory. We'll consider the starting address of the array as being memory address 1000. Recall from last week that the `int` data type uses 4 bytes of memory. In this case, the array `array` that we declared will use 10 * 4 = 40 bytes:

```
1036 |       | 10
1032 |       | 9
1028 |       | 8
1024 |       | 7
1020 |       | 6
1016 |       | 5
1012 |       | 4
1008 |       | 3
1004 |       | 2
1000 | array | 1
```

Note that I did not list `array[0]`, `array[1]`, etc. Since arrays are

stored contiguously in memory, each individual element of the array does not need to be tracked. The program only tracks the start of the array, and then adds an offset to determine the next element of the array. So, `array[0]` is saying to find the integer that is = elements array from the start of the array. `array[3]` says to find the element that is 3 elements from the start. 3 * 4 = 12, so it searches for a value that is 12 bytes from the start address. In this example, the start address is 1000, so it searches address 1012. This address stores the value 4, as expected.

Partially Filled Arrays

Sometimes, we do not know exactly how many elements an array will need to hold. This is a problem, since the size of the array must be known before the program is compiled. Therefore, a partially filled array may be needed. A partially filled array declares an array that may be too large for the number of elements it needs to hold, and then keeps track of how many elements actually get used. Let's consider this example of reading quiz scores for students. We can assume that the largest possible class has 25 students. However, the class could not full, or students could be absent when the quiz is taken. Therefore, it is possible that fewer than 25 grades need to be stored. We can use a partially filled array for this task. We'll declare an array with 25 elements, and then keep track of how many students actually have a grade. Then, we can use that value as the size of the array for the rest of the program:

```
#include <iostream>
#include <string>

int main() {
    // declare arrays of 25 strings and doubles
    // to store the students and their grades
    std::string names[25];
    double grades[25];

    // variable to track the number of students
    // that are actually entered
    int numStudents = 0;

    std::string name = "";

    std::cout << "Enter the first student's name: ";
```

```
    std::cin >> name;

    // check that "quit" was not enter for a student's name
    // also check that we have not exceeded the size of the array
    while (name != "quit" && numStudents < 24) {
        // only store the student if the name is not quit
        names[numStudents] = name;
        std::cout << "Enter grade for " << student << ": ";
        std::cin >> grades[numStudents];

        // increase the number of students
        ++numStudents;

        // prompt for the next student
        std::cout << "Enter the next student's name (or 'quit'): ";
        std::cin >> name;
    }

    std::cout << numStudents;
    std::cout << " out of 25 possible students took the quiz\n";

    // print out the students and their grades
    // instead of using the actual size of the array, 25
    // we are looping up until numStudents, which is the number
    // of indices that are actually used
    for (int i = 0; i < numStudents; ++i) {
        std::cout << names[i] << " scored "
            << grades[i] << " on the quiz\n";
    }

    return 0;
}
```

Arrays as Function Parameters

Arrays can be used as parameters to functions. Arrays are passed via "pass-by-array", which automatically passes them by reference. Therefore, if the function is not supposed to modify the array, the parameter is marked as `const`. Array parameters are specified by putting the square brackets on the parameter's name. Additionally, the size of the array should always be passed as a parameter. While there are ways to determine the size of the array within the function, there is no way to determine whether all the indices are valid, since the array could be partially filled.

```
// this function will print out an array in curly braces
// with each element separated by a comma
```

```
void printArray(const int arr[], int size) {
    bool printComma = false;
    std::cout << "{ ";
    for (int i = 0; i < size; ++i) {
        if (printComma) std::cout << ", ";
        printComma = true;
        std::cout << arr[i];
    }
    std::cout << " }";
}
```

Arrays cannot be returned from a function. Instead, if you want a function to read or modify an array, it must be passed to the function as a parameter. The following program rewrites the previous program that reads student quiz grades to use a function to read the arrays and print the arrays:

```
#include <iostream>
#include <string>

// passing the max size of the array by value
// passing numStudents by reference so the
// partially filled size is known
void readGrades(std::string names[], double grades[],
                int maxSize, int &numStudents);

// this function prints the grades with the format used in the
// original version of this program
void printGrades(std::string names[], double grades[], int numStudents

int main() {
    std::string names[];
    double grades[];
    int numStudents = 0;

    readGrades(names, grades, 25, numStudents);

    std::cout << numStudents;
    std::cout << " out of 25 possible students took the quiz\n";

    printGrades(names, grades, numStudents);

    return 0;
}

void readGrades(std::string names[], double grades[],
                int maxSize, int &numStudents) {

    std::string name = "";
```

```
    std::cout << "Enter the first student's name: ";
    std::cin >> name;

    // check that "quit" was not enter for a student's name
    // also check that we have not exceeded the size of the array
    while (name != "quit" && numStudents < maxSize - 1) {
        // only store the student if the name is not quit
        names[numStudents] = name;
        std::cout << "Enter grade for " << student << ": ";
        std::cin >> grades[numStudents];

        // increase the number of students
        ++numStudents;

        // prompt for the next student
        std::cout << "Enter the next student's name (or 'quit'): ";
        std::cin >> name;
    }
}

void printGrades(std::string names[], double grades[], int numStudents
    for (int i = 0; i < numStudents; ++i) {
        std::cout << names[i] << " scored "
            << grades[i] << " on the quiz\n";
    }
}
```

Array Bounds

The bounds of an array are the indices that are valid. Valid indices always start at 0. All C arrays are 0-indexed. This is the lower-bound of an array. The upper-bound is one less than the size of the array. A 10-element array has an upper-bound of 9. Accessing an index that is not valid for the array is called an "out-of-bounds" or "out-of-range" access. C++ provides no check if the index being accessed is out-of-bounds. Recall that an array is contiguous in memory. When accessing an element, the offset (index * data size) is added to the memory address of the first array element. This memory address is accessed. If we have a 10 element array of integers called `values` that starts at address 1000, and we try to access `values[20]`, the program will try to access 4 bytes of memory starting at 1000 + (20 * 4) = 1080. This value is not part of the array, but the program will access it like it is. This can lead to a wide range of problems. If we are reading data from `values[20]`, whatever 4 bytes of data that start at that location will be interpreted

as an integer and read as such. Obviously, this is invalid data for the array and can cause bugs later on by using an invalid value. Writing data to an invalid location can result in even worse consequences, as it can overwrite another variable or corrupt the program.

Conclusion

Arrays are fundamental data structures in computer science. Arrays allow multiple values to be stored consecutively in memory under the same name. Values are accessed via the subscript operator, which are square brackets placed after the name of the array. The bounds of the array range from 0 to one less than the size of the array. Arrays can be passed as parameters to functions with a special "pass-by-array" syntax. Pass-by-array automatically passes them by reference. Functions cannot return arrays, so functions that need to "return" an array require it to be passed as a parameter. C++ provides no checks for out-of-bounds access to array elements, so it is up to the programmer to ensure that no invalid indices are read or written to.
